5-2

一、解题过程

（1）**核心思想：二维线性插值**

已知二维规则网格点 (xi,yj) 上的函数值 zij=f(xi,yj)，构造一个二元函数 z=F(x,y)，使得 F(xi,yj)=zij，并用 F(x,y) 计算任意中间点 (x,y) 的函数值。

二维线性插值是线性插值在二维的推广，通过两次一维线性插值实现二维点的插值：先在 x 方向做两次线性插值，得到两个中间点，再在 y 方向做一次线性插值，得到目标点的插值结果。

本题的插值需求：

1. 原始网格：x 步长 400，y 步长 400，共 13×15=195 个点

2. 目标网格：x 步长 50（从 0 到 5600，共 113 个点），y 步长 50（从 0 到 4800，共 97 个点），共 97×113=10961 个点

（2）解题步骤：

1. **数据预处理**：将表格中的高程数据转换为二维数组，定义原始网格的 x,y 坐标

2. **生成目标网格**：创建 x,y 方向间隔为 50 的加密网格

3. **二维插值**：使用双线性插值计算加密网格的高程

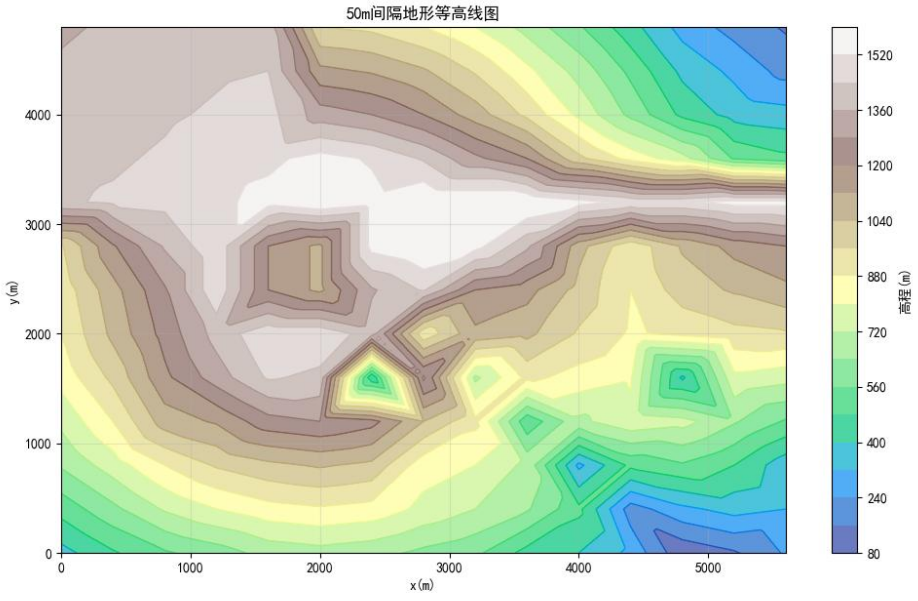
4. **绘制等高线**：用插值后的高程数据绘制等高线图，直观展示地形

二、结果呈现

1. 高程数据（间隔 50 米）Excel 文件（截取一部分数据作为示例）

	A	B	C	D	E	F
1	x(m)	y(m)	高程z(m)			
2	0	0	370			
3	50	0	382.5			
4	100	0	395			
5	150	0	407.5			
6	200	0	420			
7	250	0	432.5			
8	300	0	445			
9	350	0	457.5			
10	400	0	470			
11	450	0	480			
12	500	0	490			
13	550	0	500			
14	600	0	510			
15	650	0	520			
16	700	0	530			
17	750	0	540			
18	800	0	550			
19	850	0	556.25			
20	900	0	562.5			
21	950	0	568.75			
22	1000	0	575			
23	1050	0	581.25			
24	1100	0	587.5			
25	1150	0	593.75			
26	1200	0	600			
27	1250	0	608.75			
28	1300	0	617.5			
29	1350	0	626.25			
30	1400	0	635			
31	1450	0	643.75			
32	1500	0	652.5			
33	1550	0	661.25			
34	1600	0	670			
35	1650	0	672.5			
36	1700	0	675			

2. 等高线图



三、代码展示

```
# ==================== 导入库 ====================
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import griddata
```

```
import pandas as pd

z_raw = np.array([
    [1350,1370,1390,1400,1410,960,940,880,800,690,570,430,290,210,150],
    [1370,1390,1410,1430,1440,1140,1110,1050,950,820,690,540,380,300,210
],
    [1380,1410,1430,1450,1470,1320,1280,1200,1080,940,780,620,460,370,35
0],
    [1420,1430,1450,1480,1500,1550,1510,1430,1300,1200,980,850,750,550,5
00],
    [1430,1450,1460,1500,1550,1600,1550,1600,1600,1600,1550,1500,1500,15
50,1550],
    [950,1190,1370,1500,1200,1100,1550,1600,1550,1380,1070,900,1050,1150
,1200],
    [910,1090,1270,1500,1200,1100,1350,1450,1200,1150,1010,880,1000,1050
,1100],
    [880,1060,1230,1390,1500,1500,1400,900,1100,1060,950,870,900,936,950
],
    [830,980,1180,1320,1450,1420,400,1300,700,900,850,810,380,780,750],
    [740,880,1080,1130,1250,1280,1230,1040,900,500,700,780,750,650,550],
    [650,760,880,970,1020,1050,1020,830,800,700,300,500,550,480,350],
    [510,620,730,800,850,870,850,780,720,650,500,200,300,350,320],
    [370,470,550,600,670,690,670,620,580,450,400,300,100,150,250]
])

x_ori = np.arange(0, 5601, 400)
y_ori = np.arange(4800, -1, -400)
X_ori, Y_ori = np.meshgrid(x_ori, y_ori)

points = np.c_[X_ori.ravel(), Y_ori.ravel()]
vals = z_raw.ravel()

x_new = np.arange(0, 5601, 50)
y_new = np.arange(0, 4801, 50)
X_new, Y_new = np.meshgrid(x_new, y_new)

Z_new = griddata(points, vals, (X_new, Y_new), method='linear')

flat_x = X_new.ravel()
flat_y = Y_new.ravel()
flat_z = Z_new.ravel()

df = pd.DataFrame({
    "x(m)": flat_x,
```

```
    "y(m)": flat_y,
    "高程 z(m)": flat_z
})
df.to_csv("50m 间隔_全高程数据.csv", index=False, encoding="utf-8-sig")
print("\n 全部 50m 网格高程已保存到：50m 间隔_全高程数据.csv")

plt.figure(figsize=(12,7))
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.contour(X_new, Y_new, Z_new, levels=20, cmap="terrain",
linewidths=1)
plt.contourf(X_new, Y_new, Z_new, levels=20, cmap="terrain", alpha=0.7)
plt.colorbar(label="高程(m)")
plt.xlabel("x(m)")
plt.ylabel("y(m)")
plt.title("50m 间隔地形等高线图")
plt.grid(alpha=0.3)
plt.show()
```

5-3

一、解题过程

y=ae^bx 是非线性模型，直接用非线性最小二乘求解复杂，可以通过**对数变换**将其转化为线性模型，用**线性最小二乘（一元线性回归）**求解：

对两边取对数：lny=lna+bx

令 Y=lny，A=lna，B=b，则模型转化为**一元线性回归模型** Y=A+Bx

此时问题变为：用线性最小二乘法拟合 Y=A+Bx，求出 A,B 后，再反推 a=e^A，b=B 即可。

注：具体最小二乘计算方法参见代码部分

二、结果呈现

```
a = 11.4358, b = 0.2913
拟合公式: y = 11.4358 * e^(0.2913x)
拟合优度 R² = 0.999999
```

三、代码展示

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
```

```
from sklearn.metrics import r2_score

x = np.array([1, 2, 3, 4, 5, 6, 7, 8])
y = np.array([15.3, 20.5, 27.4, 36.6, 49.1, 65.6, 87.87, 117.6])
n = len(x)

Y = np.log(y)

sum_x = np.sum(x)
sum_Y = np.sum(Y)
sum_x2 = np.sum(x**2)
sum_xY = np.sum(x * Y)

B = (n * sum_xY - sum_x * sum_Y) / (n * sum_x2 - sum_x**2)
A = (sum_Y - B * sum_x) / n

a_linear = np.exp(A)
b_linear = B

y_linear = a_linear * np.exp(b_linear * x)
r2_linear = r2_score(y, y_linear)

print(f"a = {a_linear:.4f}, b = {b_linear:.4f}")
print(f"拟合公式: y = {a_linear:.4f} * e^({b_linear:.4f}x)")
print(f"拟合优度 R² = {r2_linear:.6f}")

x_plot = np.linspace(0, 9, 100)
y_linear_plot = a_linear * np.exp(b_linear * x_plot)

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.figure(figsize=(10, 6), dpi=100)
plt.scatter(x, y, color='red', s=60, zorder=5)
plt.plot(x_plot, y_linear_plot, color='blue', linewidth=2)

plt.xlabel('x', fontsize=12)
plt.ylabel('y', fontsize=12)
plt.title('最小二乘法拟合', fontsize=14, pad=20)
plt.grid(linestyle='--', alpha=0.3)
plt.legend(fontsize=10)
plt.tight_layout()
plt.show()
```

5-4

一、解题过程

（1）原理分析

根据水箱内液体的质量平衡方程，单位时间内水箱质量的变化量等于流入质量与流出质量之差:

$$\frac{dm}{dt} = Q_{in}(\mathrm{t})\text{-}\mathrm{f}(\mathrm{t})$$

其中：m(t)=ρSh(t) 为 t 时刻水箱内水的质量，$Q_{in}(t)$为水泵灌水速度。根据假设，泵水期间 $Q_{in}(t)$ = Q（常数），非泵水期间$Q_{in}(t)$= 0。f(t)为待求的任意时刻流出流量。

（2）核心假设与约束

1. 流量平滑性：假设公众用水需求 f(t)是关于时间的连续平滑曲线，不会因为水泵的启停而发生瞬时突变。

2. 非负约束：流出流量必须满足 f(t)≥0。

（3）解题步骤

第一步：非泵水时段流量采样

在不泵水的观测区间内，$Q_{in}(\mathrm{t})$=0，此时段的平均流出流量可通过水位的变化直接计算为$\frac{dm}{dt}$，之后将计算得到的多个平均流量点视为该区间中点时刻的瞬时流量样本。

第二步：全时段流量曲线拟合

利用第一步得到的流量样本点，采用平滑样条插值拟合出全天 24 小时的流量函数 f(t)。

注：不直接拟合水位 h(t)是为了避免在泵水切换点产生数值震荡（吉布斯现象），通过拟合流量样本可以保证 f(t)的全局平滑性。

第三步：基于质量守恒求解水泵流量 Q

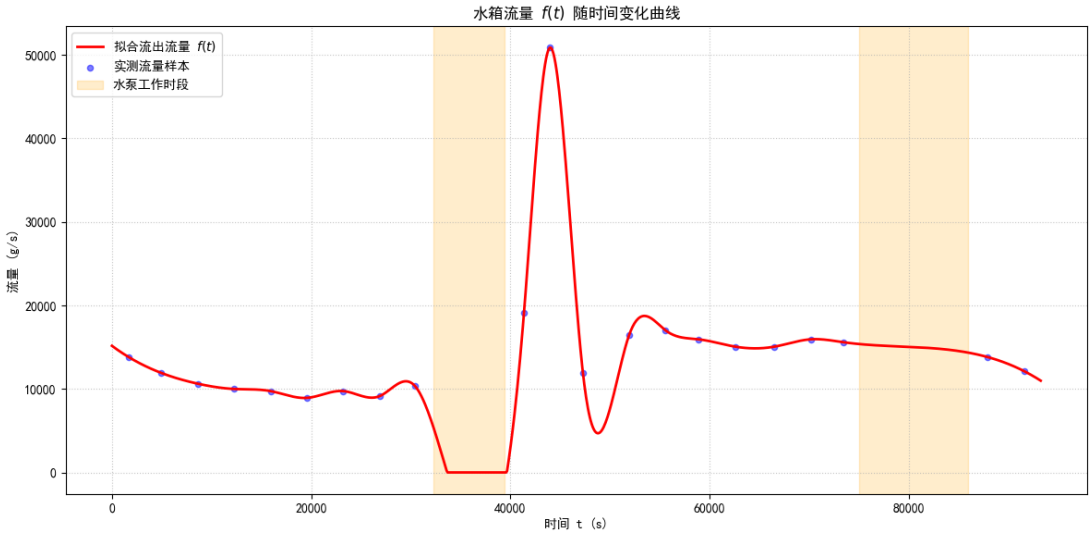
在泵水时段$[\mathrm{t_{start}}, \mathrm{t_{end}}]$内，水箱质量的变化量由灌入量和流出量的积分共同决定:

$$m(t_{end}) - m(t_{start}) = \int_{t_{start}}^{t_{end}} (Q - f(t))\, dt$$

由于 Q 是常数，将其提出来并变换公式，得到 Q 的精确估计值:

$$Q = \frac{[m(t_{end}) - m(t_{start})] + \int_{t_{start}}^{t_{end}} f(t)\, dt}{t_{end} - t_{start}}$$

二、结果呈现



水泵恒定灌水速度 Q = 84702.24 g/s

	A	B	C	D	E
1	时间t(s)	流量f(t)(g/s)			
2	0	15162.99			
3	100	15074.73			
4	200	14987.53			
5	300	14901.39			
6	400	14816.3			
7	500	14732.24			
8	600	14649.22			
9	700	14567.22			
10	800	14486.24			
11	900	14406.26			
12	1000	14327.29			
13	1100	14249.3			
14	1200	14172.3			
15	1300	14096.27			
16	1400	14021.21			
17	1500	13947.11			
18	1600	13873.96			
19	1700	13801.75			
20	1800	13730.48			
21	1900	13660.13			
22	2000	13590.7			
23	2100	13522.18			
24	2200	13454.57			
25	2300	13387.85			
26	2400	13322.01			
27	2500	13257.05			
28	2600	13192.97			
29	2700	13129.74			
30	2800	13067.37			
31	2900	13005.85			
32	3000	12945.16			
33	3100	12885.3			
34	3200	12826.26			
35	3300	12768.04			
36	3400	12710.62			

(导出的 Excel 部分示例数据展示)

三、代码展示

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import UnivariateSpline
import pandas as pd

E = 30.24
D = 57 * E
S = np.pi * (D/2)**2
rho = 1.0

raw_data = [
    (0, 3175), (3316, 3110), (6635, 3054), (10619, 2994), (13937, 2947),
    (17921, 2892), (21240, 2850), (25223, 2795), (28543, 2752), (32284,
2697),
    # 泵水时段 1: 32284 到 39435
    (39435, 3550), (43318, 3445), (44636, 3350), (49953, 3260), (53936,
3167),
    (57254, 3087), (60574, 3012), (64554, 2927), (68535, 2842), (71854,
2767),
    (75021, 2697),
    # 泵水时段 2: 75021 到 85968
    (85968, 3475), (89953, 3397), (93270, 3340)
]

t_pts = np.array([x[0] for x in raw_data])
h_pts = np.array([x[1] for x in raw_data]) * 0.01 * E
m_pts = rho * S * h_pts

pump_idx = [(9, 10), (20, 21)]

f_samples_t = []
f_samples_v = []

for i in range(len(t_pts)-1):
    is_pumping = any(i == start for start, end in pump_idx)

    if not is_pumping:
        dt = t_pts[i+1] - t_pts[i]
        dm = m_pts[i+1] - m_pts[i]
        avg_f = -dm / dt
        f_samples_t.append((t_pts[i] + t_pts[i+1]) / 2)
        f_samples_v.append(avg_f)
```

```
spline_f = UnivariateSpline(f_samples_t, f_samples_v, s=0.5, k=3)

def get_f(t):
    val = float(spline_f(t))
    return max(val, 0)

t_s, t_e = t_pts[9], t_pts[10]
m_s, m_e = m_pts[9], m_pts[10]

t_fine = np.linspace(t_s, t_e, 100)
f_integral = np.trapz([get_f(t) for t in t_fine], t_fine)

Q = (m_e - m_s + f_integral) / (t_e - t_s)
print(f"水泵恒定灌水速度 Q = {Q:.2f} g/s")

t_all = np.linspace(0, 93270, 1000)
f_all = [get_f(t) for t in t_all]

t_list = np.arange(0, 93271, 100)
df = pd.DataFrame({
    "时间 t(s)": t_list,
    "流出流量 f(t)(g/s)": [round(get_f(t), 2) for t in t_list]
})
df.to_excel("水箱流量结果.xlsx", index=False)

plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False
plt.figure(figsize=(12, 6))

plt.plot(t_all, f_all, 'r-', linewidth=2, label="拟合流出流量 $f(t)$")
plt.scatter(f_samples_t, f_samples_v, color='blue', s=20, alpha=0.5,
label="实测流量样本")

for start, end in pump_idx:
    plt.axvspan(t_pts[start], t_pts[end], color='orange', alpha=0.2,
label="水泵工作时段" if start==9 else "")

plt.title("水箱流量 $f(t)$ 随时间变化曲线")
plt.xlabel("时间 t (s)")
plt.ylabel("流量 (g/s)")
plt.legend()
plt.grid(True, linestyle=':', alpha=0.7)
plt.tight_layout()
plt.show()
```